Motion Planning

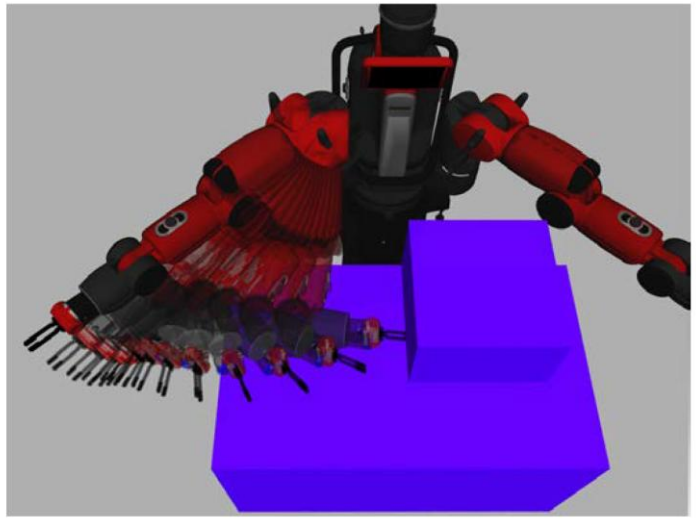
CS 6341 Robotics

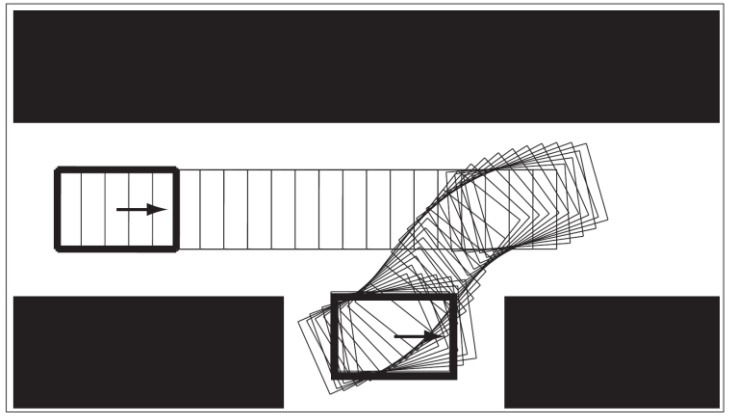
Instructor Yu Xiang

The University of Texas at Dallas

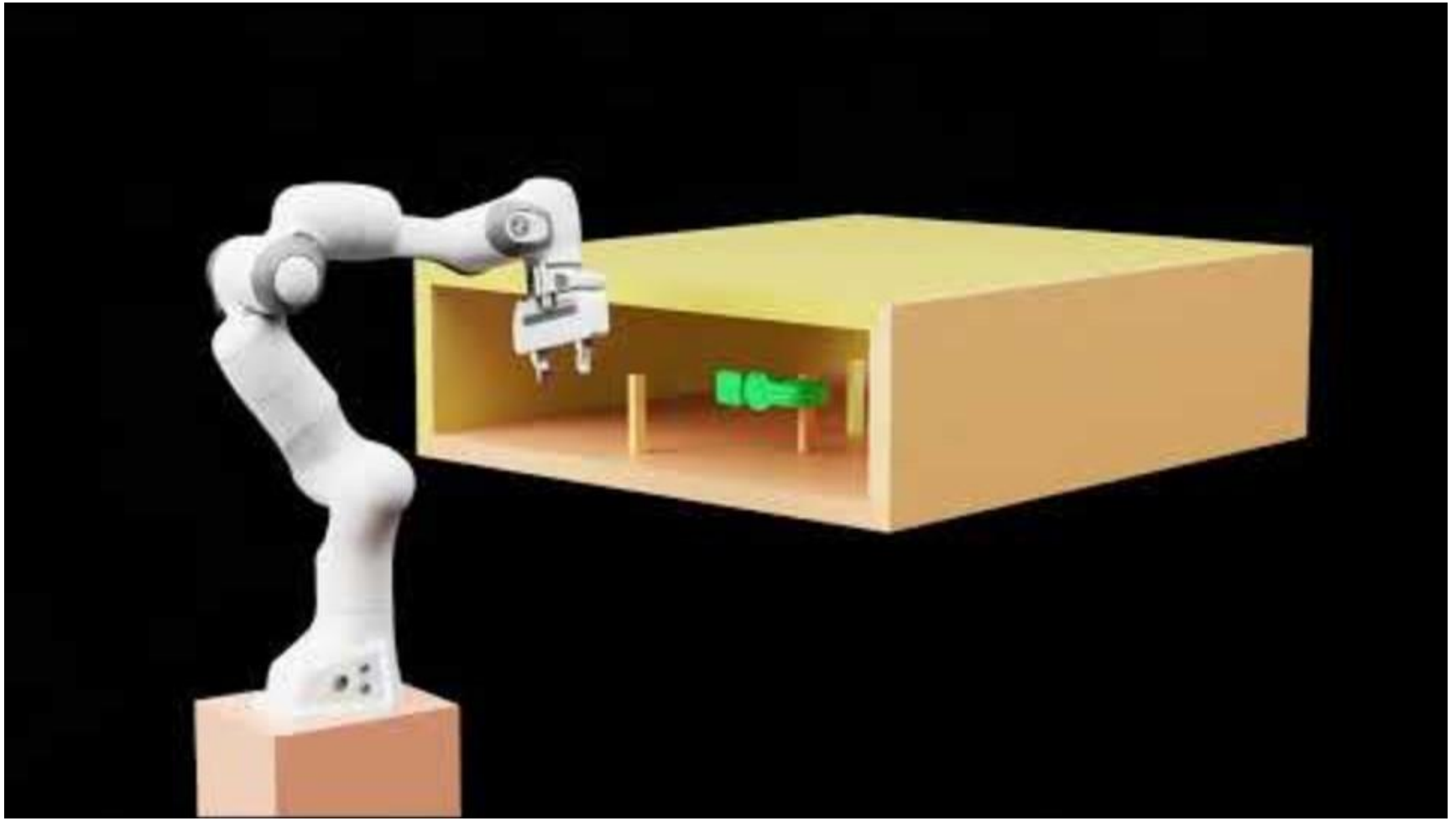
Motion Planning

- Motion planning: finding a robot motion from a start state to a goal state (A to B)
  - Avoids obstacles
  - Satisfies other constraints such as joint limits or torque limits





Example: cuRobo from NVIDIA



https://developer.nvidia.com/blog/cuda-accelerated-robot-motion-generation-in-milliseconds-with-curobo/

Motion in Configuration Space

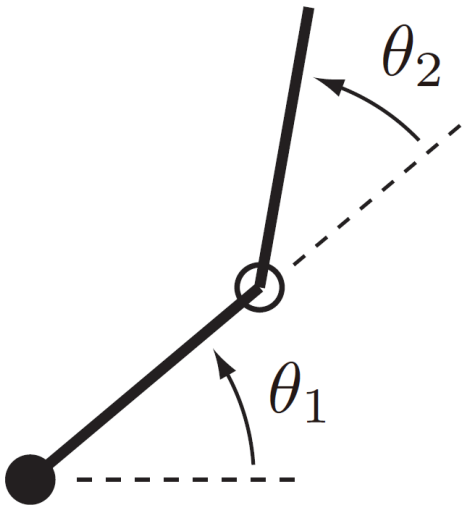
- The configuration of a robot arm with n joints
  - n joint positions $q = (\theta_1, \ldots, \theta_n)$

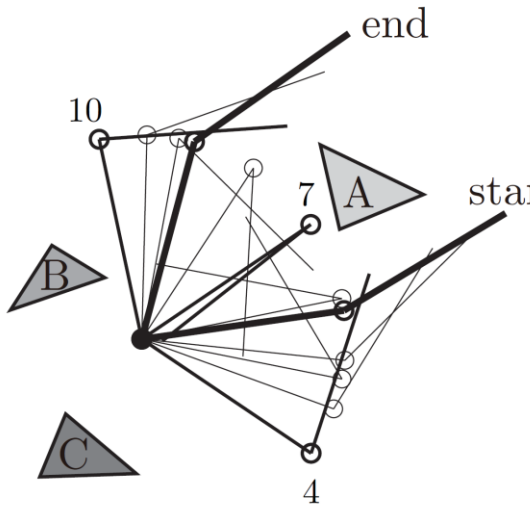
- Free C-space $\mathcal{C}_{\text{free}}$
  - Configurations where the robot neither penetrates an obstacle nor violated a joint limit

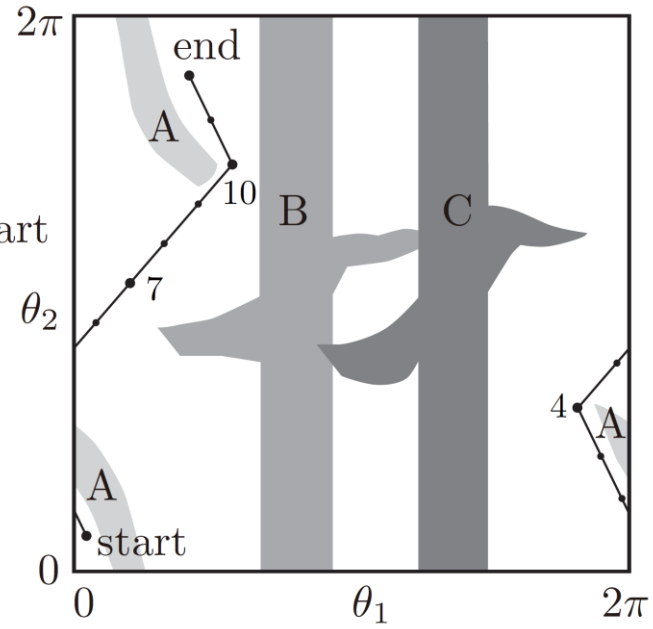
- Obstacles in C-space $\mathcal{C}_{\text{obs}}$ $\mathcal{C} = \mathcal{C}_{\text{free}} \cup \mathcal{C}_{\text{obs}}$
  - Joint limits are treated as obstacle in the configuration space

Configuration Space Obstacles

- A 2R planar arm







Configuration space

Distance to Obstacles in Configuration Space

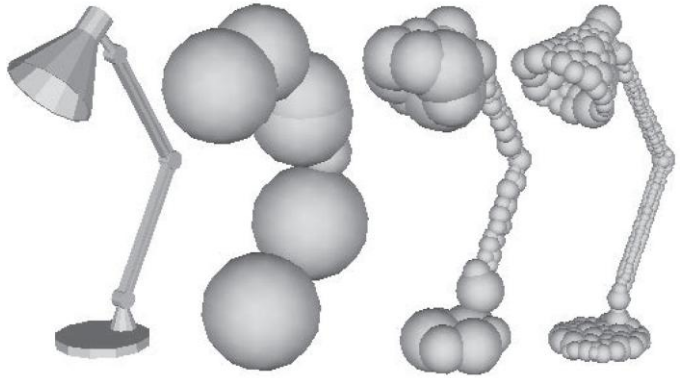
- Given a C-obstacle $\mathcal{B}$ and a configuration q, the distance between a robot and the obstacle

$$
\begin{aligned}
d(q, \mathcal{B}) &> 0 \qquad (\text{no contact with the obstacle}),\\
d(q, \mathcal{B}) &= 0 \qquad (\text{contact}),\\
d(q, \mathcal{B}) &< 0 \qquad (\text{penetration}).
\end{aligned}
$$

- A distance measurement algorithm determines $d(q, \mathcal{B})$
- A collision detection algorithm determines whether $d(q, \mathcal{B}_i) \leq 0$

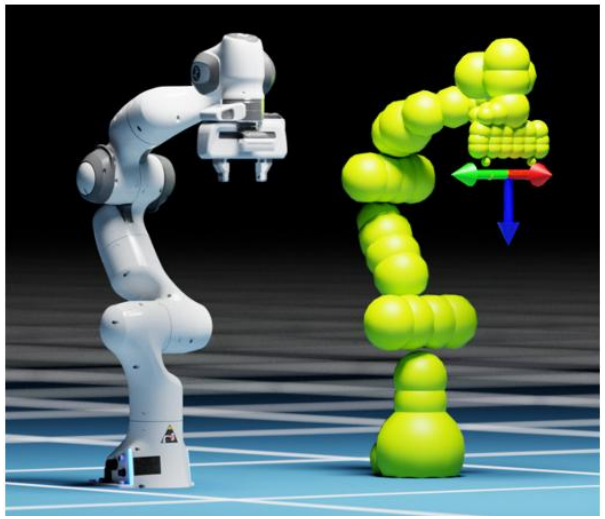
Distance to Obstacles

- Approximation of 3D shapes using 3D spheres



- Robot: k spheres of radius R_i centered at $r_i(q)$
- Obstacle: l spheres of radius B_j centered at b_j
- The distance between the robot and the obstacle

$$d(q, \mathcal{B}) = \min_{i,j} \|r_i(q) - b_j\| - R_i - B_j$$



cuRobo

Motion Planning

- Find collision-free path

$$q(s), s \in [0, 1]$$

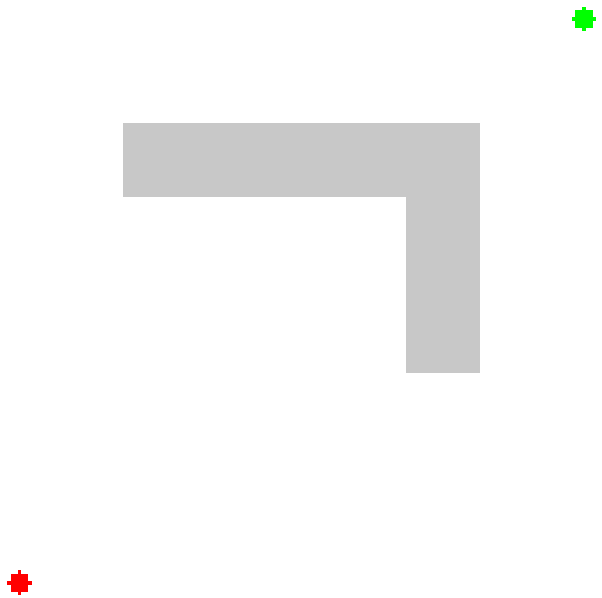
$$q(0) = q_{\text{start}}$$

$$q(1) = q_{\text{goal}}$$

How to obtain q_goal?

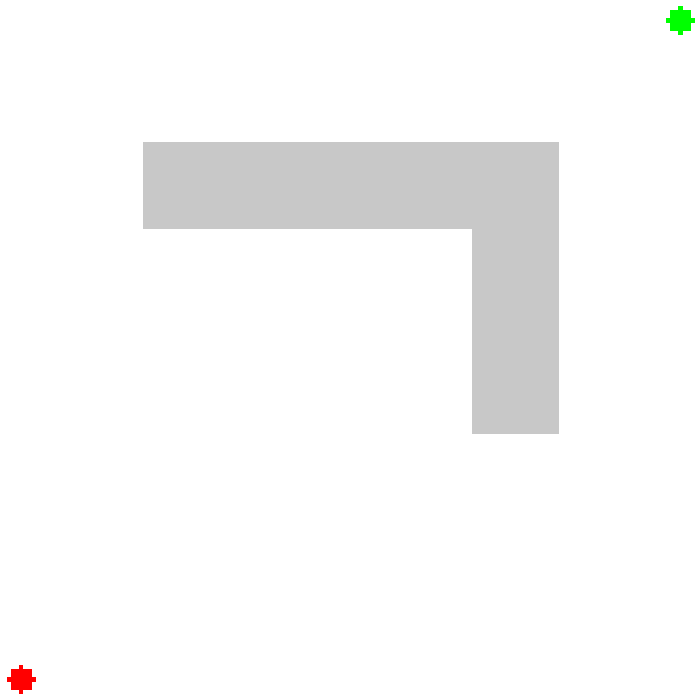
2D Path Planning

Dijkstra's Algorithm



https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm

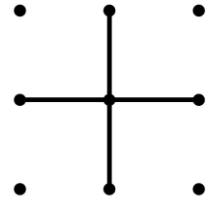
A* Search Algorithm



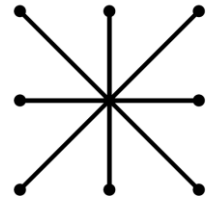
https://en.wikipedia.org/wiki/A*_search_algorithm

C-Space Path Planning: Grid Methods

- Discretize the configuration space into a grid
  - If the C-space is n dimension, we use k grid points along each dimension
  - The C-space is represented by k^n grid points

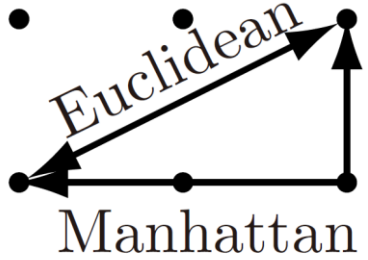


4-connected



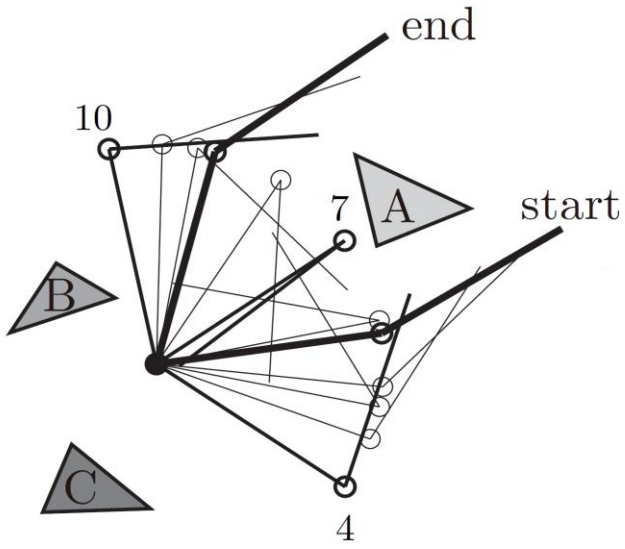
8-connected

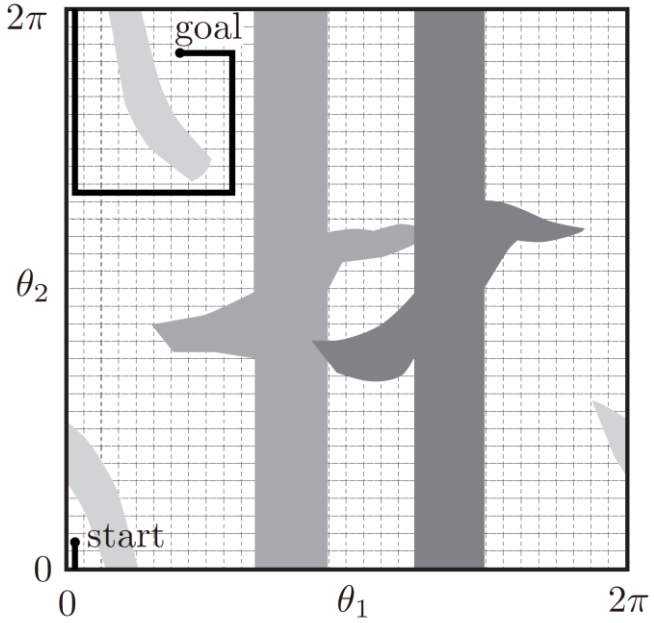
- We can apply the A* search algorithm for path planning with a C-space grid



C-Space Path Planning: Grid Methods

- A* grid-based path planner





$k = 32$

Grid-based path planning is only suitable for low-dimensional C-space

Number of grid points k^n

```
>>> np.power(32, 7.0)
34359738368.0
```

C-Space Path Planning: Sampling Methods

- Sampling methods
  - Randomly or deterministically sampling the C-space or state-space to find the motion plan
  - Give up resolution-optimal solutions of a grid search, quickly find solutions in high-dimensional state space
  - Most sampling methods are probabilistically complete: the probability of finding a solution, when one exists, approaches 100% as the number of samples goes to infinity

Rapidly exploring Random Trees (RRTs)

Algorithm 10.3 RRT algorithm.

1: initialize search tree T with x_{start}
2: **while** T is less than the maximum tree size **do**
3: $x_{\text{samp}} \leftarrow$ sample from $\mathcal{X}$
4: $x_{\text{nearest}} \leftarrow$ nearest node in T to x_{samp}
5: employ a local planner to find a motion from x_{nearest} to x_{new} in the direction of x_{samp}
6: **if** the motion is collision-free **then**
7: add x_{new} to T with an edge from x_{nearest} to x_{new}
8: **if** x_{new} is in $\mathcal{X}_{\text{goal}}$ **then**
9: **return** SUCCESS and the motion to x_{new}
10: **end if**
11: **end if**
12: **end while**
13: **return** FAILURE

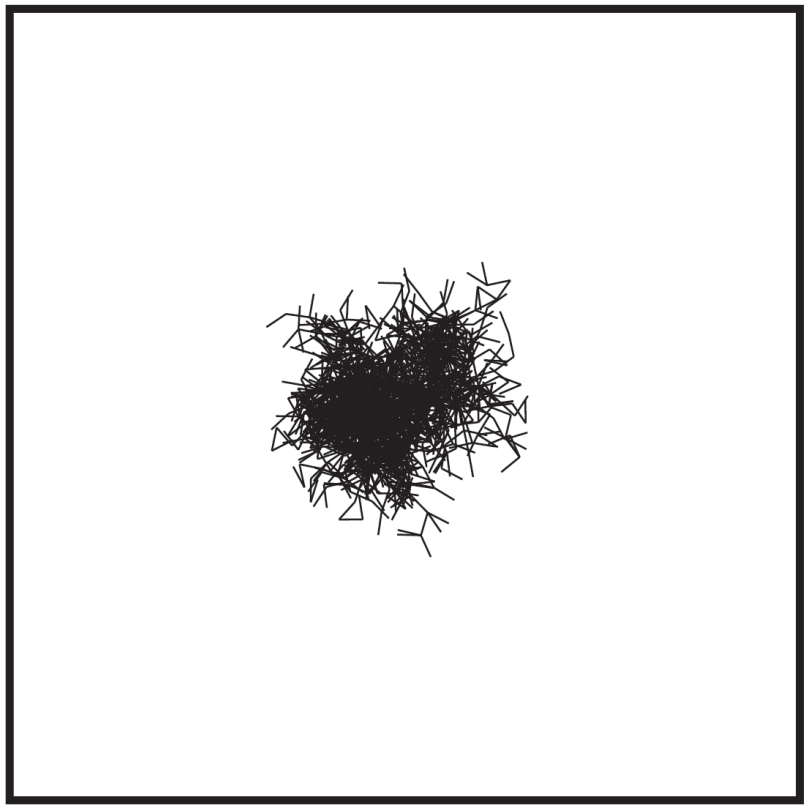
kinematic problems

$$x = q$$

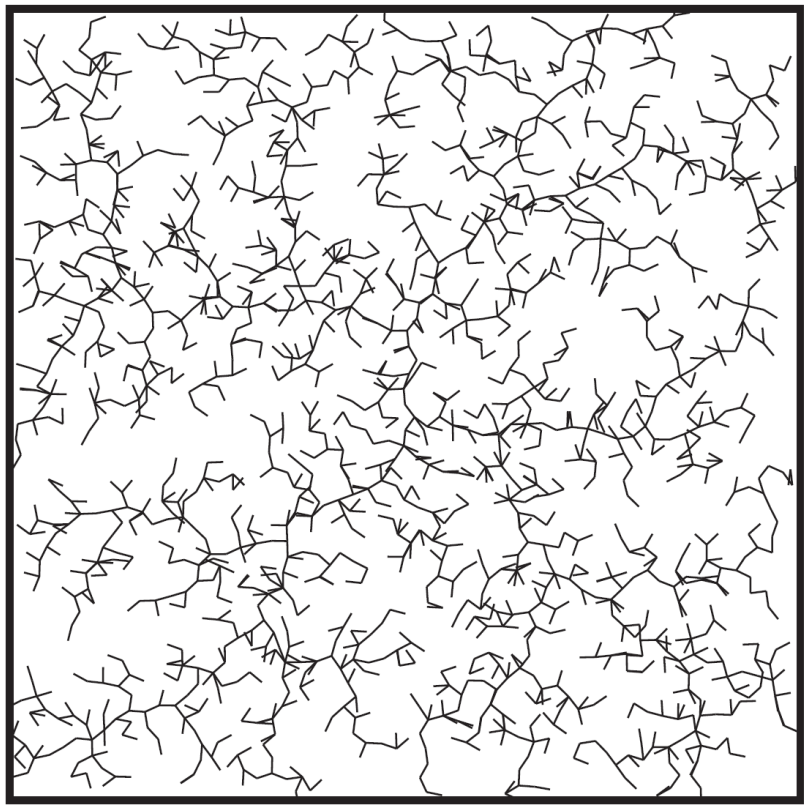
- Line 3, uniform sampling with a bias towards goal
- Line 4, Euclidean distance
- Line 5, use a small distance d from, check collision along the line

x_{nearest} on the straight line to x_{samp}

Rapidly exploring Random Trees (RRTs)



A tree generated by applying a uniformly-distributed random motion from a randomly chosen tree node does not explore very far.



2000 nodes

A tree generated by the RRT algorithm

Rapidly exploring Random Trees (RRTs)

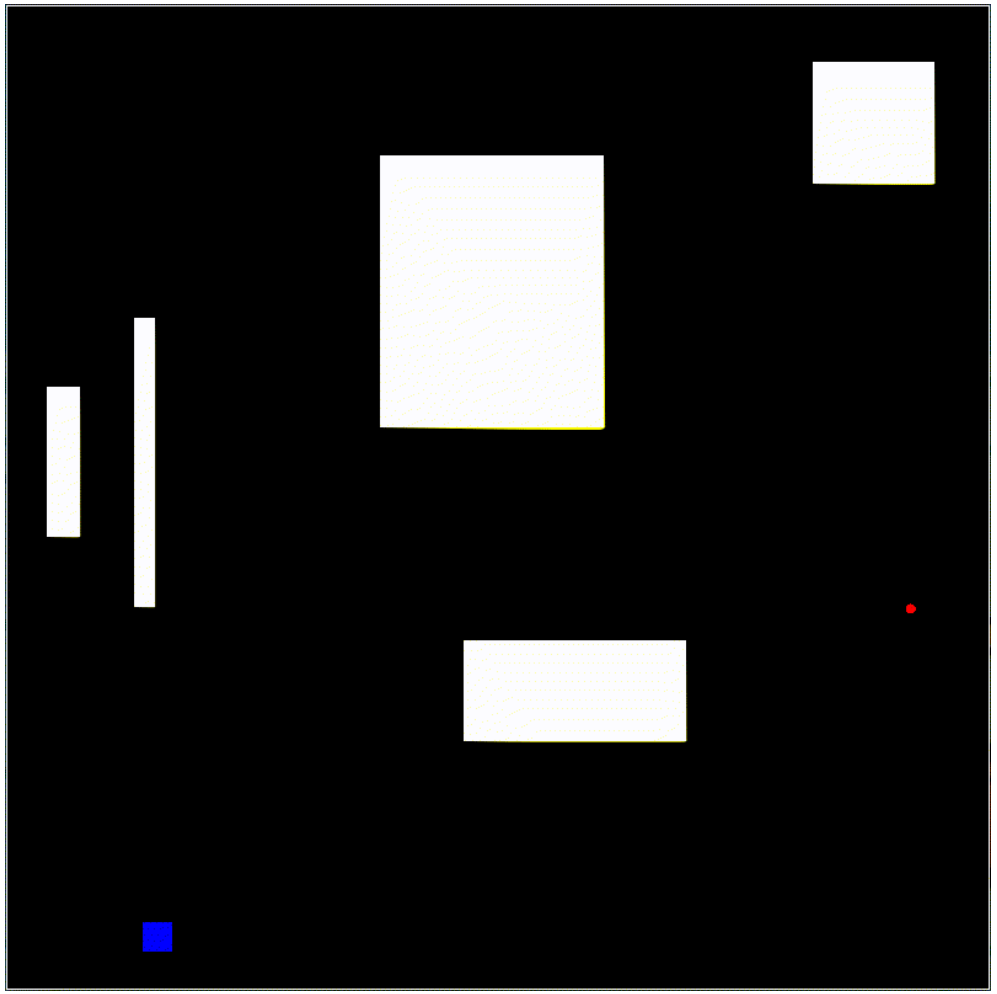
An animation of an RRT starting from iteration 0 to 10000

https://en.wikipedia.org/wiki/Rapidly-exploring_random_tree

Rapidly exploring Random Trees (RRTs)

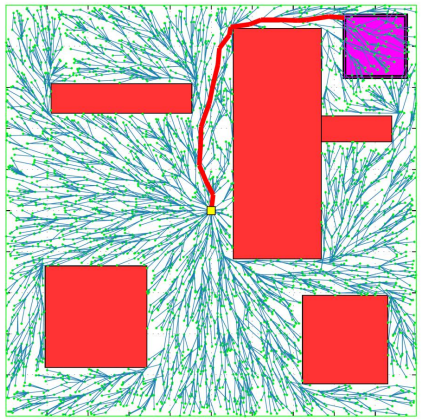
- Bidirectional RRT
  - Grows two trees, one forward from x_{start} , one backward from x_{goal}
  - Alternating between growing the two trees x_{samp}
  - Trying to connect the two trees by choosing x_{goal} from the other tree
  - Con: faster, can reach the exact goal
  - Pro: the local planer might not be able to connect the two trees

Bidirectional RRT



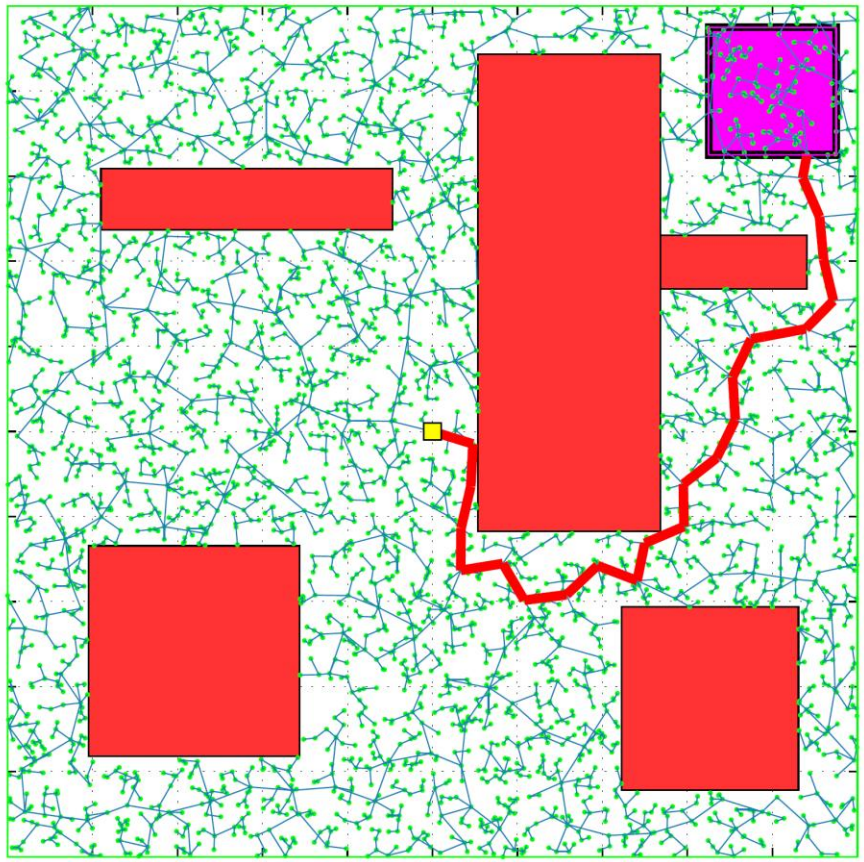
https://github.com/JakeInit/RRT

RRT*

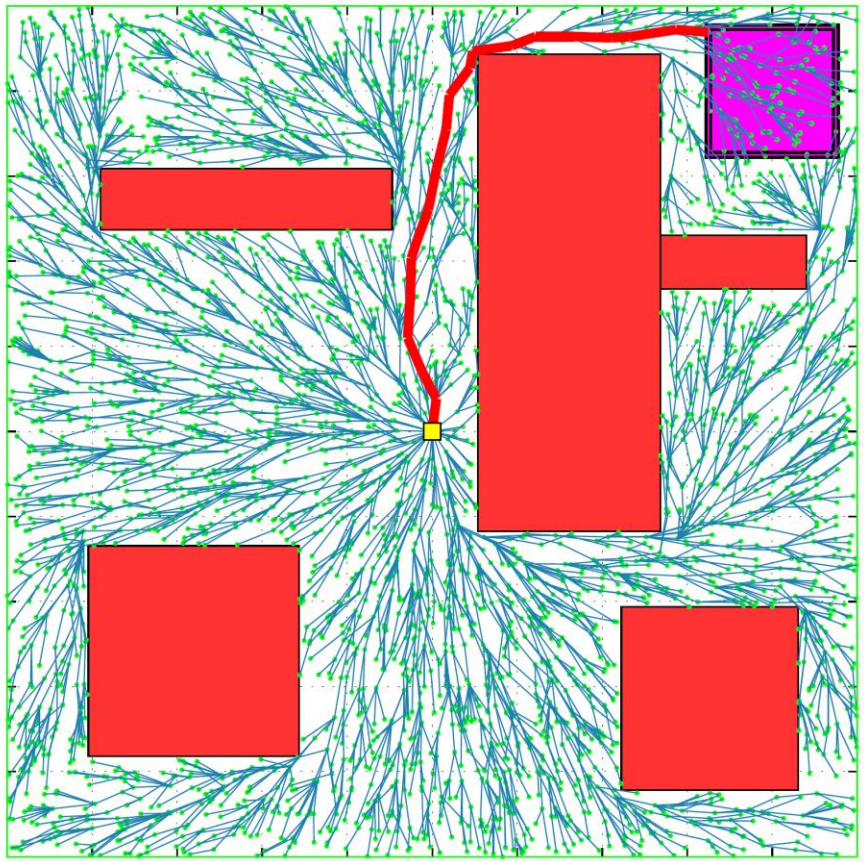


- RRT*
  - Continually rewires the search tree to ensure that it always encodes the shortest path from x_{start} to each node in the tree
  - To insert x_{new} to the tree, consider $x \in \mathcal{X}_{\text{near}}$ sufficiently near to x_{new}
    - Collision free
    - Minimizes the total cost from x_{start} to x_{new}
  - Consider each $x \in \mathcal{X}_{\text{near}}$ to see whether it could be reached at lower cost by a motion through x_{new} , change the parent of x to x_{new} (rewiring)

RRT vs. RRT*



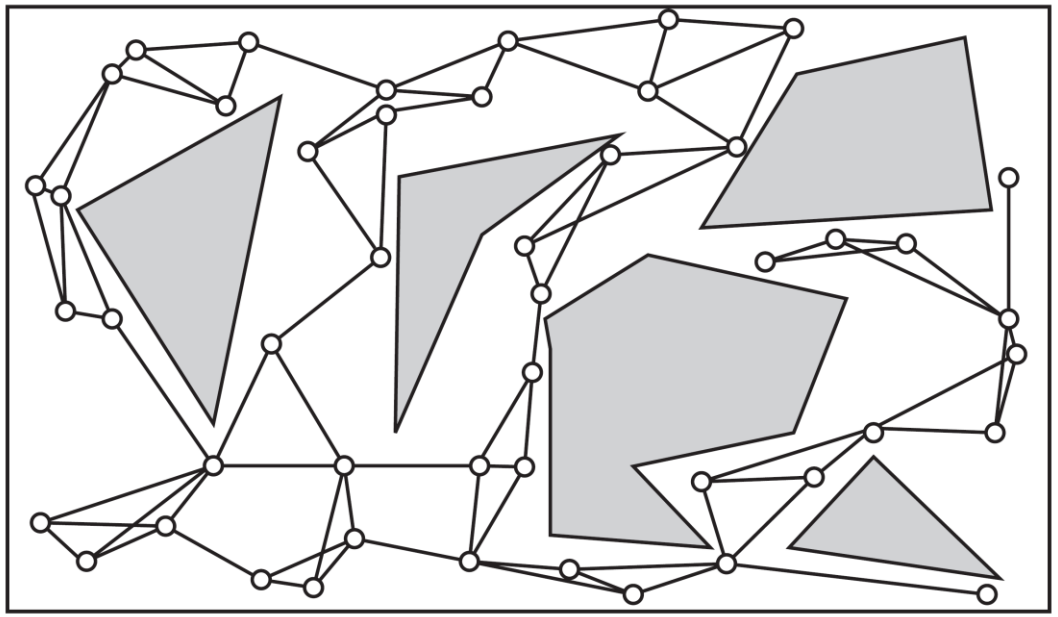
RRT



RRT*

Probabilistic Roadmaps (PRMs)

- PRM uses sampling to build a roadmap representation of $\mathcal{C}_{\text{free}}$
- Connect a start node q_{start} and a goal node q_{goal} to the roadmap
- Search for a path, e.g., using A*

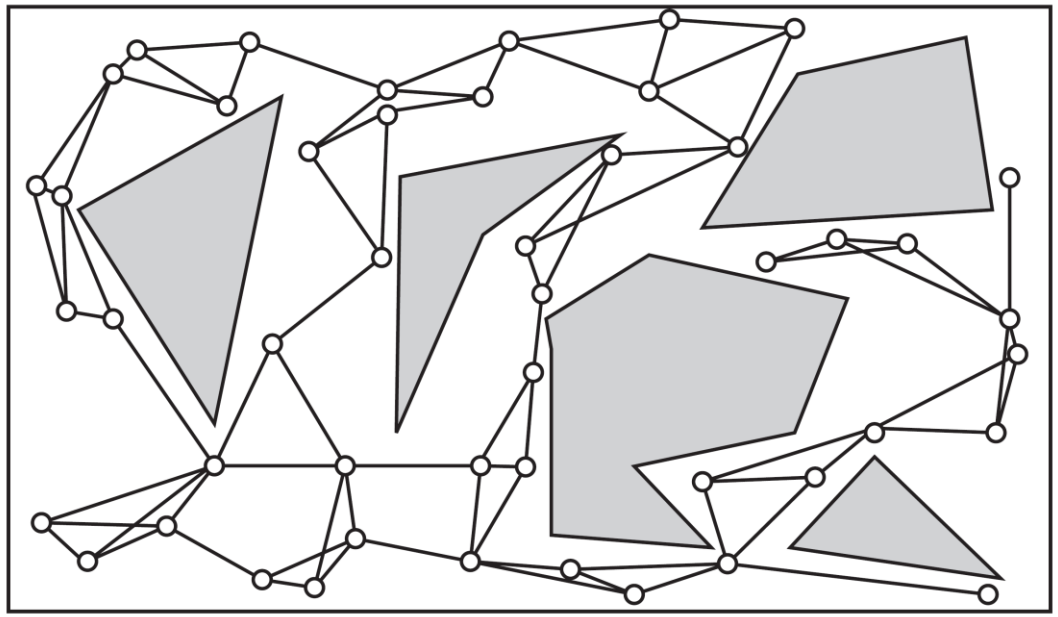


Probabilistic Roadmaps (PRMs)

- PRM uses sampling to build a roadmap representation of $\mathcal{C}_{\text{free}}$

Algorithm 10.4 PRM roadmap construction algorithm (undirected graph).

```
for i = 1, ..., N do
    q_i ← sample from C_free
    add q_i to R
end for
for i = 1, ..., N do
    N(q_i) ← k closest neighbors of q_i
    for each q ∈ N(q_i) do
        if there is a collision-free local path from q to q_i and
        there is not already an edge from q to q_i then
            add an edge from q to q_i to the roadmap R
        end if
    end for
end for
return R
```



Optimization for Motion Generation

- CuRobo optimization

Reaching an end-effector pose

$$
\begin{aligned}
\underset{\theta_{[1,T]}}{\arg\min} \quad & C_{\text{task}}(X_g, \theta_T) + \sum_{t=1}^{T} C_{\text{smooth}}(\cdot) \\
\text{s.t.,} \quad & \theta^- \preceq \theta_t \preceq \theta^+, \forall t \in [1, T] \\
& \dot{\theta}^- \preceq \dot{\theta}_t \preceq \dot{\theta}^+, \forall t \in [1, T] \\
& \ddot{\theta}^- \preceq \ddot{\theta}_t \preceq \ddot{\theta}^+, \forall t \in [1, T] \\
& \dddot{\theta}^- \preceq \dddot{\theta}_t \preceq \dddot{\theta}^+, \forall t \in [1, T] \\
& \dot{\theta}_T, \ddot{\theta}_T, \dddot{\theta}_T = 0 \\
& C_r(K_s(\theta_t)) \preceq 0, \forall t \in [1, T] \\
& C_w(K_s(\theta_t)) \preceq 0, \forall t \in [1, T]
\end{aligned}
$$

Self-collision

Workspace collision



Summary

- Overview of motion planning
  - Configuration space obstacle
  - Distance to obstacles

- Path Planning
  - Grid methods
  - Sampling methods
  - Optimization

Further Reading

- Chapter 10 in Kevin M. Lynch and Frank C. Park. Modern Robotics: Mechanics, Planning, and Control. 1st Edition, 2017.